DataScientest
MLOps bootcamp · cohort JUL26

PROJECT REPORT

Weather MLOps

An end-to-end machine-learning operations system for next-day rain prediction at 49 Australian weather stations

AUTHORS

Gabriel Marchesan Almeida
Jonathan Vints
Thomas Maisch
Ziad Al-Haj Hemidi

MENTOR

Nicolas Fradin

Defended on 15 October 2026
github.com/DataScientest-Studio/jul26_bmlops_int_weather · commit 2063257

Abstract

This report describes **Weather MLOps**, an end-to-end machine-learning operations system that predicts whether it will rain tomorrow at 49 Australian weather stations.

An XGBoost classifier is trained on the Kaggle WeatherAUS dataset (145,460 daily station records, November 2007 to June 2017), extended with data from the Open-Meteo archive API in the same 23-column schema. The model is served over an authenticated HTTPS API: Nginx is the only public entry point and FastAPI loads the model that holds the MLflow `@champion` alias.

Every evening an Airflow pipeline ingests new weather, rebuilds time-ordered training, validation and test splits, registers the dataset version, labels earlier predictions with the observed outcome and runs Evidently to measure feature drift and delayed model performance. When at least 30 predictions have a known outcome and their recall falls below 0.75, or their ROC-AUC drops more than 0.02 below the champion's, a Grafana alert starts a retraining DAG. A new model replaces the champion only if it passes a promotion gate: a complete and clean release, validation recall of at least 0.75 and a higher validation ROC-AUC than the current champion. Data are versioned with DVC and SHA-256 manifests, models with MLflow, and lineage, predictions and secrets live in Supabase.

With its baseline settings, the champion reached a ROC-AUC of 0.866 and a recall of 0.776 on 90 unseen validation days, against a majority-class baseline that is 74.7 % accurate on the current test window but never predicts rain. The gate has already rejected weaker candidates without manual intervention. An experiment on Singapore data shows how distribution shift hurts the model: ROC-AUC falls to 0.696 over all Singapore days and to 0.641 on Singapore 2022–2024, and retraining recovers only part of the loss. This is the case for monitoring the model after deployment.

Keywords: MLOps, rain prediction, XGBoost, MLflow, Airflow, DVC, Evidently, drift monitoring.

Acknowledgements

We thank our mentor, Nicolas Fradin, for his guidance and feedback throughout the project, and DataScientest for the bootcamp that made it possible.

Contents

1 Introduction 1

1.1 Problem 1
1.2 Objectives 1
1.3 Data 1
1.4 Tools 2
1.5 Team 2
1.6 Structure of this report 2

2 System overview 3

2.1 Runtime 4

3 Architecture 5

3.1 Whole system 5
3.2 Components 6
3.3 Compose services 6
3.4 MLflow server 8
3.5 Request path 9
3.6 Source map 9

4 Training and serving pipeline 11

4.1 Runtime loop 11
4.2 Data build 12
4.3 Train / serve 12
4.4 Makefile 12
4.5 Known caveat: Open-Meteo timeout 13

5 Data versioning 14

5.1 Identities 14
5.2 Register a snapshot 15
5.3 Source files 15

6 Model 17

6.1 Hyperparameters 17
6.2 Split 18
6.3 Historical metrics (old 70 / 15 / 15 split) 18
6.4 Retrain 18

7 Model registry and promotion 19

7.1 What a training run records 19
7.2 First champion 20

7.3 Lifecycle 21
7.4 Promotion rule 22
7.5 What the API returns 22
7.6 DVC copy 22
7.7 Source files 22

8 Orchestration with Airflow 23
8.1 Components 23
8.2 Catch-up 23
8.3 DAG stages 24
8.4 Security boundary 25
8.5 Networks 25
8.6 Running the stack 25
8.7 Local-only secrets 26
8.8 What Airflow does not do yet 26
8.9 Source files 26

9 Monitoring 27
9.1 Flow 28
9.2 Metrics 28
9.3 Evidently job 29
9.4 Alerts 29
9.5 Dashboards 30
9.6 Source files 32

10 Database schema 33
10.1 Tables, buckets and functions 33
10.2 Secrets 34
10.3 Changing the schema 34

11 Results and discussion 35
11.1 Evaluation protocol 35
11.2 Model performance 35
11.3 Distribution shift: Australia to Singapore 36
11.4 Engineering results 37

12 Limitations and future work 38
12.1 Known limitations 38
12.2 Future work 38

13 Conclusion 40

References 41

Appendix A Setup 42
A.1 Requirements 42

A.2 Install 42
A.3 Airflow 43

Appendix B Reproduction 44
B.1 CI 45

Appendix C Glossary 46

List of figures

2.1 Runtime 4
3.1 Whole system 5
3.2 MLflow server 8
3.3 Request path 9
4.1 Runtime loop 11
5.1 Register a snapshot 15
7.1 Lifecycle 21
9.1 Flow 28
9.2 Grafana dashboard: Weather API 30
9.3 Grafana dashboard: Weather Model 31
9.4 Grafana alert rules 32

List of tables

3.1 Components of the system and what each one does 6
3.2 Compose services 6
3.3 MLflow server 8
3.4 Source map 9
4.1 Data build 12
4.2 Train / serve 12
4.3 Makefile 12
5.1 Data versioning 14
5.2 Identities 14
5.3 Source files (Data versioning) 15
6.1 Hyperparameters 17
6.2 Split 18
6.3 Historical metrics (old 70 / 15 / 15 split) 18
7.1 What a training run records 19
7.2 Lifecycle of a model version: steps and outputs 21
7.3 Source files (Model registry and promotion) 22
8.1 Files of the Airflow stack 23
8.2 Schedule, catch-up and retries of the two DAGs 23

8.3 Tasks of the daily weather_pipeline DAG 24
8.4 Tasks of the on-demand weather_retrain DAG 24
8.5 Source files (Orchestration with Airflow) 26
9.1 Metrics 28
9.2 Evidently checks: drift warning and retraining rule 29
9.3 Where each Evidently run writes its results 29
9.4 Source files (Monitoring) 32
10.1 Tables, buckets and functions 33
11.1 Model performance 35
11.2 Distribution shift: Australia to Singapore 36
12.1 Known limitations 38
A.1 Install 42
C.1 Glossary 46

CHAPTER 1

Introduction

1.1 Problem

Predicting whether it will rain tomorrow at a weather station is a binary classification problem with imbalanced classes. Across the Kaggle history, 22.4 % of the labelled station-days are followed by a rainy day; in the current 90-day test window it is 1,116 of 4,410 station-days (25.3 %). A model that always answers "no rain" is therefore 74.7 % accurate on that window and completely useless: it never catches a rainy day. The metrics we optimise and the thresholds we enforce follow from this imbalance.

1.2 Objectives

The aim of the project was not a single good model, but a system that keeps a model correct over time without manual steps:

- **Reproducible data:** every model can be traced back to the exact data and code that produced it.
- **Tracked experiments:** every training run records its parameters, metrics, lineage and artifacts.
- **Controlled promotion:** a new model reaches users only after it passes an explicit quality gate.
- **Secure serving:** a single authenticated HTTPS entry point, with rate limits and secrets kept out of the code.
- **Monitoring and automated retraining:** drift and real performance are measured every day, and a measured loss of quality starts retraining automatically.

1.3 Data

The labelled history is the Kaggle **WeatherAUS** dataset [1], originally from the Australian Bureau of Meteorology: 145,460 daily rows from 49 stations, November 2007 to June 2017, with 23 columns (morning and afternoon readings, daily extremes, rainfall, wind and the `RainTomorrow` label).

To go beyond 2017 we call the **Open-Meteo archive API** [2] at each station's coordinates and convert the answer into the same 23 columns. Morning and afternoon values come from the 9:00 and 15:00 hourly readings; cloud cover is converted from percent to oktas, sunshine from seconds

to hours and wind direction from degrees to 16 compass points. `RainTomorrow` is true when the next day's rain is at least 1.0 mm. The DVC snapshot of 2 October 2026 holds 185,150 rows from the 49 stations.

1.4 Tools

The model is an XGBoost classifier [3] inside a scikit-learn pipeline [4]. Experiments and the model registry use MLflow [5], data versioning uses DVC [6], orchestration uses Apache Airflow [7], drift monitoring uses Evidently [8], and service monitoring uses Prometheus and Grafana [9].

1.5 Team

The project was built by Gabriel Marchesan Almeida, Jonathan Vints, Thomas Maisch and Ziad Al-Haj Hemidi during the DataScientest MLOps bootcamp (cohort JUL26), and defended on 15 October 2026.

1.6 Structure of this report

Chapter 2 gives an overview of the system and Chapter 3 its architecture. Chapters 4 to 10 follow the data and the model through the system: the training and serving pipeline, data versioning, the model, the model registry, orchestration with Airflow, monitoring and the database schema. Chapter 11 presents the results, Chapter 12 the limitations and future work, and Chapter 13 concludes, followed by the references. The appendices contain the setup instructions, the reproduction commands and a glossary.

CHAPTER 2

System overview

This chapter gives a first view of the running system: which components exist, how a prediction request travels through them and how monitoring can start retraining. Chapter 3 then describes each component in detail.

The system predicts `RainTomorrow` at an Australian weather station. Git holds the code, dataset files live in Supabase Storage and Postgres serves as the catalog. FastAPI serves the model behind Nginx, the only public HTTPS entry point, and MLflow acts as the model registry.

The app runtime is **Docker Compose**. DVC stays on the host. Airflow is a separate stack under `airflow/` — see Chapter 8 (Orchestration with Airflow). Prometheus, Grafana and Evidently close the loop — see Chapter 9 (Monitoring).

2.1 Runtime

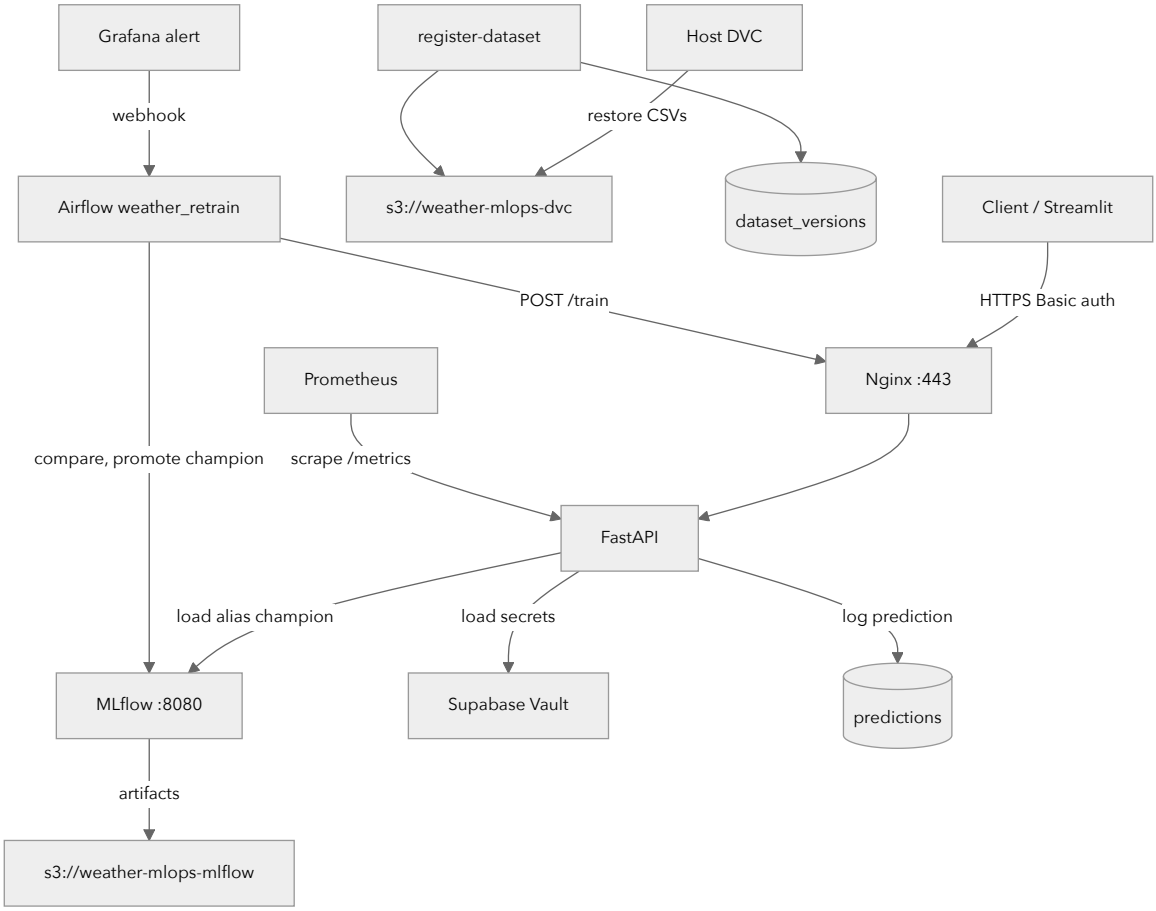


Figure 2.1 Runtime

CHAPTER 3

Architecture

The system is built from a small number of containerised services, each with one responsibility. This chapter describes them, how they connect and which ports they expose, in support of the secure-serving and traceability objectives of Section 1.2.

Compose is the only app runtime. There is no trainer container. Training is `POST /train` on the API, reached through Nginx.

3.1 Whole system

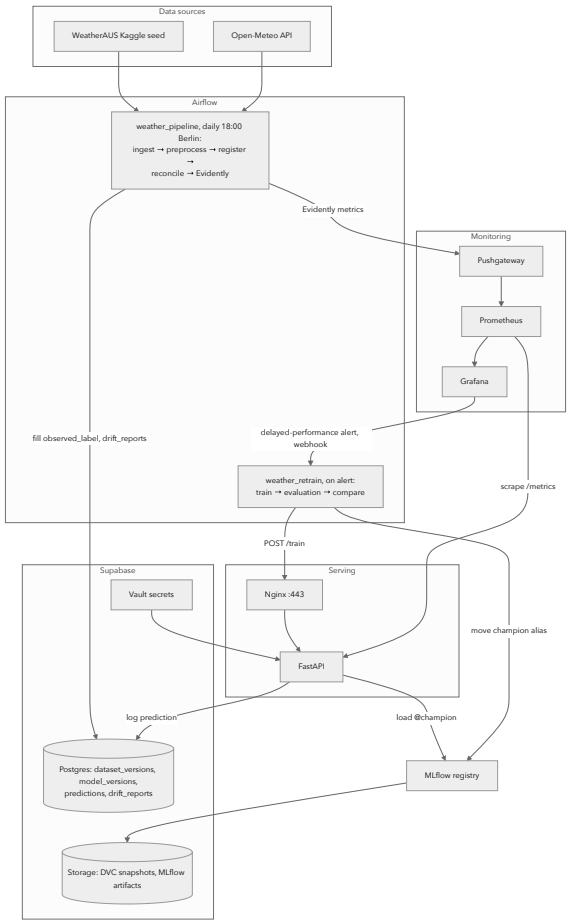


Figure 3.1 Whole system

The closed loop: every prediction is logged in `predictions` → the daily DAG fills `observed_label` once the real next-day weather is ingested → Evidently computes delayed performance → Grafana alerts → webhook starts `weather_retrain` → `compare` moves `@champion` only if the gate passes → the API picks up the new alias on the next request. Details in Chapter 9 (Monitoring) and Chapter 8 (Orchestration with Airflow).

3.2 Components

Table 3.1 Components of the system and what each one does

Piece	Path	Role
FastAPI	`src/weather_mlops/api/main.py`	`/health`, `/predict`, `/predict/live_data`, `/train`, `/monitor/simulate` (demo), `/metrics` (internal)
HTTP Basic	Vault → API process env	Protects predict and train. `/health` stays open
Nginx	`docker/nginx/`	TLS, HTTP→HTTPS, per-IP rate limits. Public edge. `/metrics` → 404
MLflow	`docker/mlflow/`	Tracking + registry on `127.0.0.1:8080`. Not behind Nginx
Streamlit	`src/weather_mlops/demo/`	Demo app and the slides for the project defense. Calls Nginx. No `SUPABASE_KEY`
DVC	host `make dvc-*`	Restore dataset blobs. Not the production catalog
Postgres	`supabase/schema.sql`	Observations, dataset + model catalog, predictions, drift reports
Storage	buckets `weather-mlops-dvc`, `weather-mlops-mlflow`	Immutable snapshots + MLflow artifacts
Prometheus + Pushgateway	`docker/prometheus/`	Scrape API every 5 s; receive Evidently batch metrics
Grafana	`docker/grafana/`	2 dashboards, 7 alerts, retrain webhook. All provisioned as code
Evidently	`src/weather_mlops/monitoring/drift.py`	Feature drift (warning) + delayed performance (retrain). Each run logged to MLflow experiment `weather-monitoring`

3.3 Compose services

Seven project-built images (`docker/*/Dockerfile`: ingestion, preprocess, api, nginx, mlflow, demo, test) plus the Prometheus, Pushgateway and Grafana images: 11 services.

Table 3.2 Compose services

Service	Type	Profile
`ingestion`	job	default
`preprocess`	job	default
`api`	long-running, internal	default
`nginx`	long-running, `:80` / `:443`	`gateway`
`mlflow`	long-running, `127.0.0.1:8080`	`mlflow`
`streamlit`	long-running, `127.0.0.1:8501`	`streamlit`
`test`	job	`test`
`baseline`	job, first champion only (`make baseline`)	`baseline`
`prometheus`	long-running, `127.0.0.1:9090`	`monitoring`
`pushgateway`	long-running, `127.0.0.1:9091`	`monitoring`
`grafana`	long-running, `127.0.0.1:3000`	`monitoring`
Airflow stack	separate Compose under `airflow/`	see Chapter 8 (Orchestration with Airflow)

`make up` / `make api` start MLflow, wait for a healthy API, then put Nginx in front. `make monitoring` adds Prometheus, Pushgateway and Grafana. `make airflow-up` brings up the orchestrator on its own Docker network; the webserver and the scheduler also join `weather_network`, so DAG containers reach nginx/api and Grafana reaches the Airflow REST API.

3.4 MLflow server

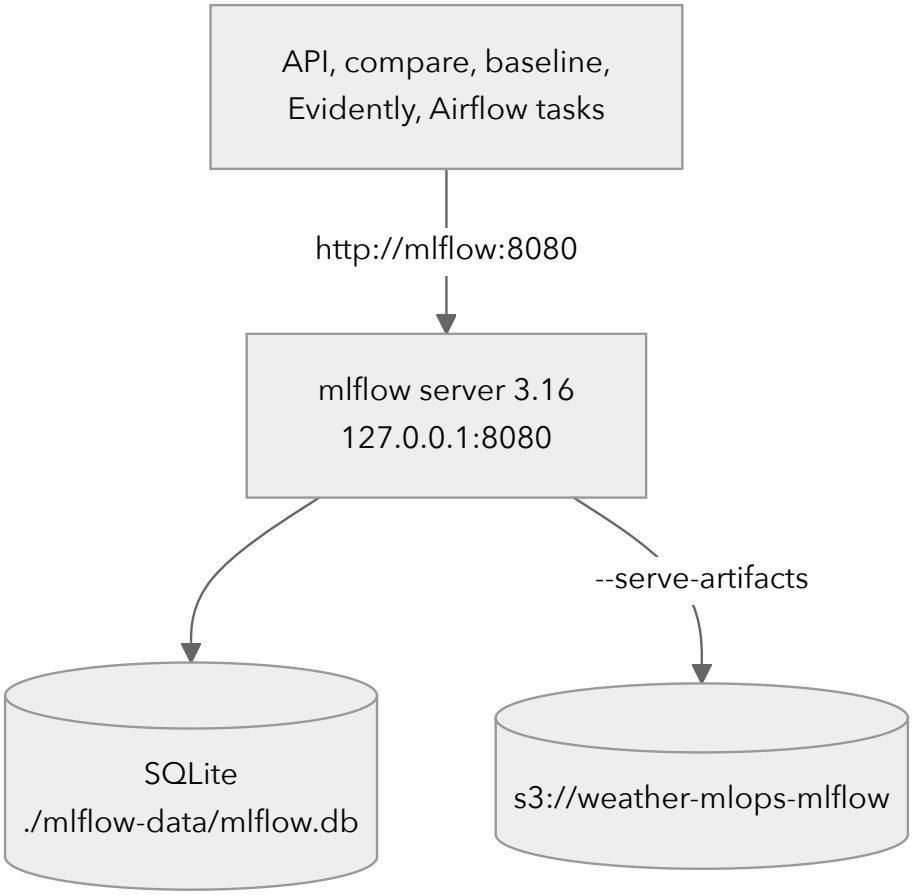


Figure 3.2 MLflow server

Table 3.3 MLflow server

Part	Where
Runs, metrics, tags, registry, aliases	SQLite `./mlflow-data/mlflow.db`
Artifacts (models, reports, manifests)	`s3://weather-mlops-mlflow`
S3 keys	`mlflow` container only
Access	`127.0.0.1:8080` on the host, `mlflow:8080` in Compose

Clients never need S3 keys, only `MLFLOW_TRACKING_URI`. Compose and the Airflow DAG set it to `http://mlflow:8080`; unset, nothing is tracked.

Two experiments: `weather-rainfall-classifier` for training runs (Chapter 7 (Model registry and promotion)) and `weather-monitoring` for Evidently runs (Chapter 9 (Monitoring)).

3.5 Request path

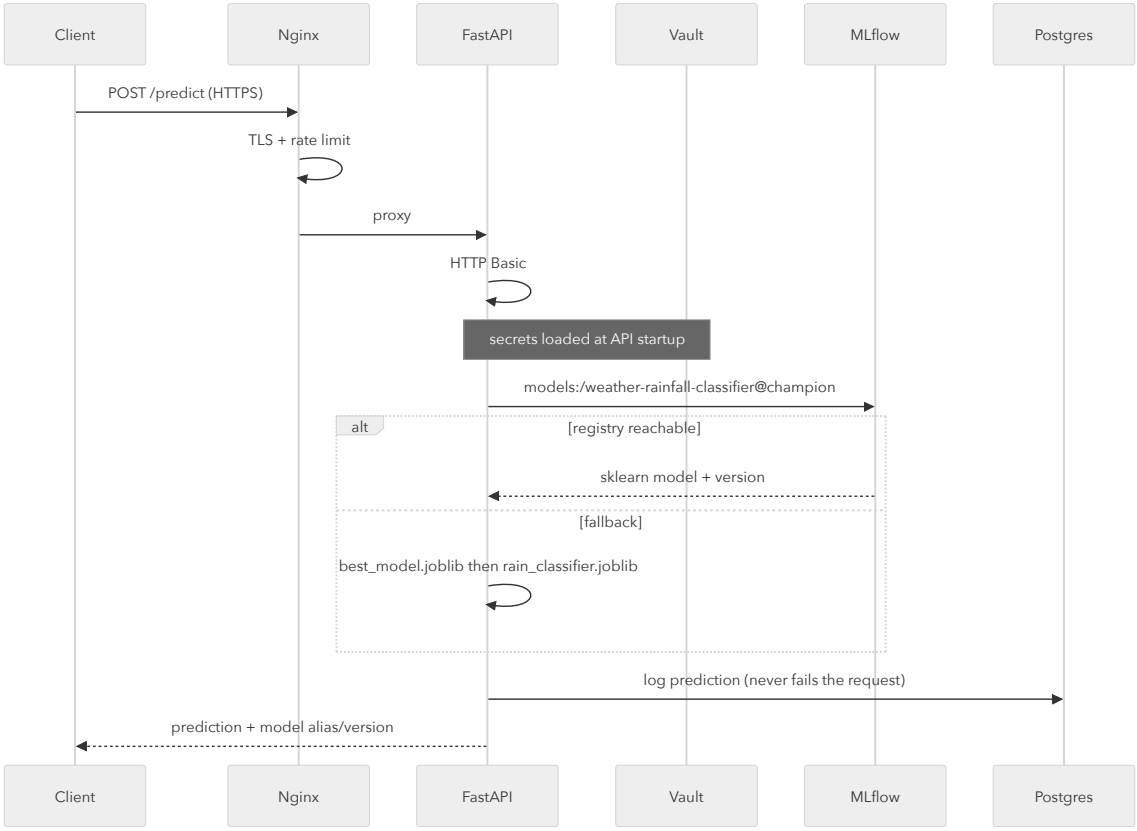


Figure 3.3 Request path

3.6 Source map

Table 3.4 Source map

Concern	Module
Endpoints	`src/weather_mlops/api/main.py`
API metrics	`src/weather_mlops/api/metrics.py`
Local TLS skip (self-signed Nginx)	`src/weather_mlops/api/http.py`
Champion load	`src/weather_mlops/models/predict.py`
Register run	`src/weather_mlops/models/tracking.py`
Promote alias	`src/weather_mlops/models/comparison.py`
First champion	`src/weather_mlops/models/baseline.py`
Catalog upsert, prediction log	`src/weather_mlops/data/database.py`
Outcome reconciliation	`src/weather_mlops/data/outcomes.py`

Evidently	`src/weather_mlops/monitoring/drift.py`
Vault	`src/weather_mlops/security/` + `scripts/with_vault_env.py`

CHAPTER 4

Training and serving pipeline

This chapter follows data and models through the system, from raw files to a served prediction, and shows how the manual commands relate to the scheduled Airflow runs.

Daily runtime is Compose, not `dvc repro`. DVC stages still exist for publishing pointer hashes (`make dvc-repro`). Training is never a container startup step.

4.1 Runtime loop

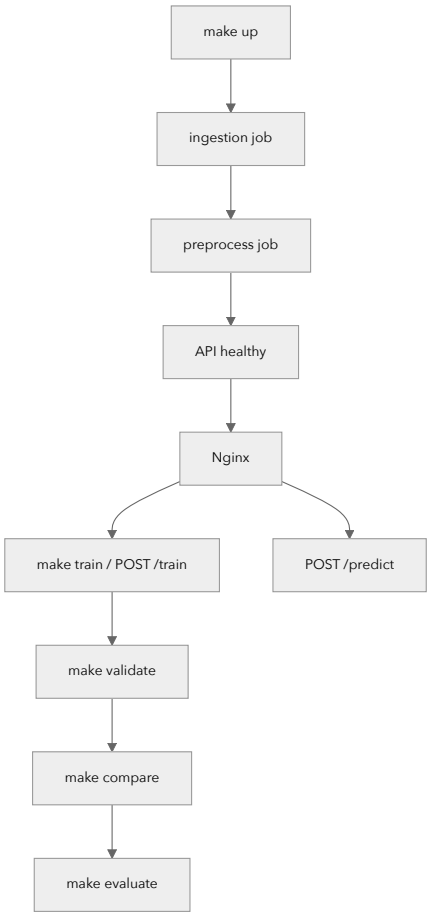


Figure 4.1 Runtime loop

`make pipeline` is `train → validate → compare → evaluate`.

4.2 Data build

Table 4.1 Data build

Step	How	Writes
Fill gap after Kaggle seed	`make backfill-open-meteo` (once, resumable, 90-day chunks)	`data/raw/incremental/`
Merge seed + Open-Meteo	ingestion job / `merge_raw`	`data/raw/weatherAUS_current.csv`
Version raw file	`version_raw`	`data/metadata/weatherAUS.json`
Splits	preprocess job	`data/processed/X_*.csv`, `y_*.csv`, `manifest.json`
Catalog	`make register-dataset`	Storage snapshot + `dataset_versions`

Splits are by date, with the newest data as test; the window sizes are given in Chapter 6 (Model).

4.3 Train / serve

Table 4.2 Train / serve

Step	How	Writes
Train	`POST /train` (`src/weather_mlops/models/training.py`)	`models/rain_classifier.joblib`, train metrics, MLflow candidate
Validate	`src/weather_mlops/models/evaluation.py`	`reports/metrics/validation.json`
Promote	`src/weather_mlops/models/comparison.py`	alias `champion`, `best_model.joblib`
Evaluate	same evaluation module, test split	`reports/metrics/evaluation.json`
Predict	API or `scripts/predict_via_api.py`	JSON through Nginx

The DVC stage `predict_sample` only writes a sample file; real predictions always go through Nginx.

4.4 Makefile

Table 4.3 Makefile

Task	Command
Ingest + preprocess + API + Nginx + MLflow	`make up`

API + Nginx (data already there)	`make api`
Retrain + promote + test metrics	`make pipeline`
Restore blobs	`make dvc-config && make dvc-pull`
Tests, incl. live HTTPS + rate-limit (HTTP 429)	`make test`
Scheduled daily DAG	`make airflow-up` then unpause `weather_pipeline` and `weather_retrain` – see Chapter 8 (Orchestration with Airflow)
Prometheus + Grafana	`make monitoring` – see Chapter 9 (Monitoring)
Evidently on host	`make evidently`
First champion from Kaggle	`make baseline` – see Chapter 7 (Model registry and promotion)

On a schedule, Chapter 8 (Orchestration with Airflow) runs these steps for you.

4.5 Known caveat: Open-Meteo timeout

The daily fetch in `src/weather_mlops/data/open_meteo.py` defaults to `timeout_seconds=30`. The archive endpoint can take close to a minute, so `ReadTimeout` happens now and then. The DAG retries 3× with a 1-minute delay. Backfill uses 60 s. Both paths retry HTTP 429 (rate limit).

CHAPTER 5

Data versioning

Every model must lead back to the exact data that trained it. This chapter explains the three layers that make this possible: DVC for the files, Postgres for the catalog and SHA-256 manifests that tie them together.

Data versioning relies on three layers with distinct roles, which are not interchangeable.

Table 5.1 Data versioning

Layer	What it stores	Who uses it
Git	code + DVC pointer files	everyone
DVC (`no_scm`)	CSV/joblib blobs in `s3://weather-mlops-dvc`	laptop restore (`make dvc-pull`)
Postgres `dataset_versions`	sha256 identity + storage URI + git commit	production lineage

DVC is not the production catalog. The catalog is Postgres.

DVC tracks the Kaggle seed, `data/raw/incremental/` (Open-Meteo backfill), `data/raw/open_meteo_Singapore/` (drift fixture), the stage outputs in `dvc.yaml` (merged raw, splits, metrics) and the served model `models/best_model.joblib`.

5.1 Identities

Table 5.2 Identities

Object	Identity	Notes
Observation	`(date, location)`	PK on `weather_observations`
Dataset snapshot	`(dataset_name, sha256)`	PK on `dataset_versions`
Processed training set	combined CSV sha256 in `data/processed/manifest.json`	what `/train` logs as `dataset_sha256`

Same sha256 uploaded twice is a no-op. Storage keys look like `raw/<sha>/...` and `processed/<sha>/...`.

5.2 Register a snapshot

```
make register-dataset
```

Default `--kind all` registers two rows in `dataset_versions`:

- `raw`: sha256 of `data/raw/weatherAUS_current.csv`
- `processed`: one combined hash of the six split CSVs (from `manifest.json`). This is the dataset ID training saves as `dataset_sha256`.

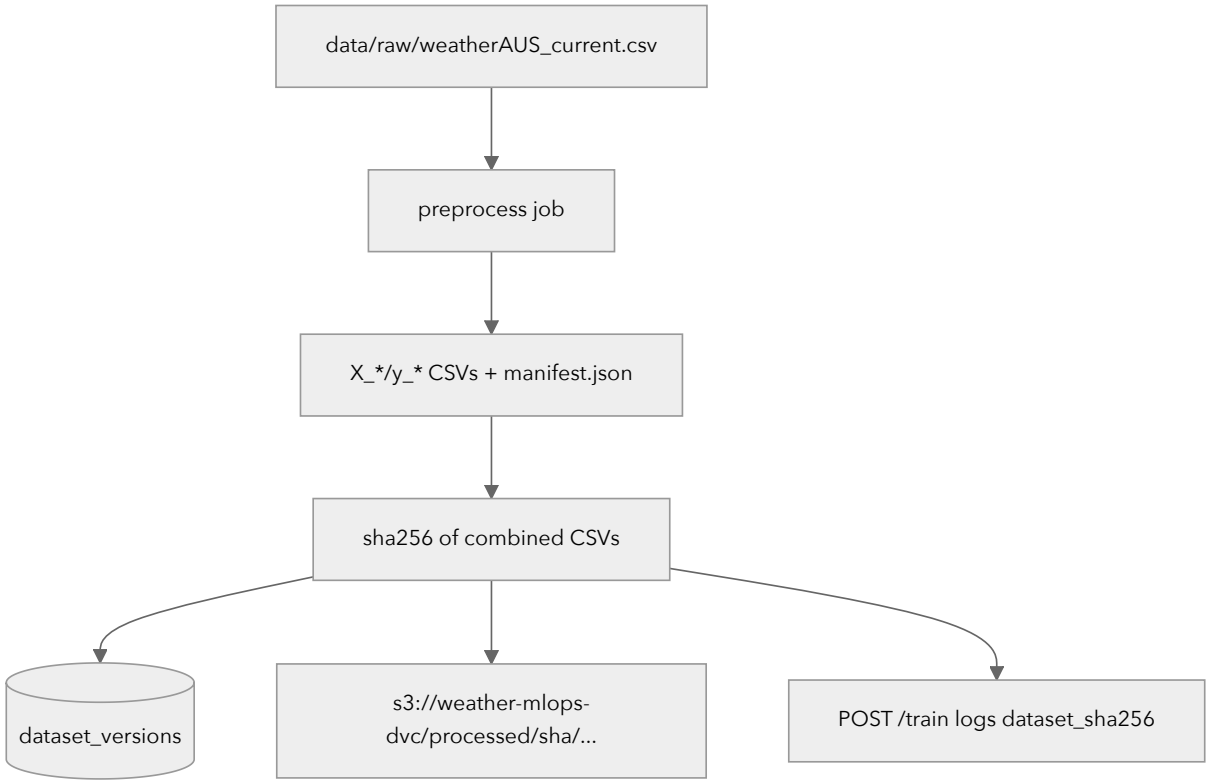


Figure 5.1 Register a snapshot

5.3 Source files

Table 5.3 Source files (Data versioning)

Step	Call
Hash + metadata JSON	`src/weather_mlops/data/versioning.py`
Verify splits match manifest	`src/weather_mlops/data/manifest.py` `verify_processed_manifest`
Upload + upsert	`scripts/register_dataset_version.py` → `store_dataset_metadata`
Host restore	`make dvc-pull`

`store_dataset_metadata` rejects a conflicting git commit / parent / storage URI for the same `(dataset_name, sha256)`. It keeps the first path, size, and md5.

CHAPTER 6

Model

This chapter describes the classifier itself: preprocessing, hyperparameters and the chronological split used to evaluate it.

The model is an XGBoost classifier, trained in `src/weather_mlops/models/training.py`.

Preprocessing: scikit-learn `ColumnTransformer`, in `src/weather_mlops/data/preprocess.py`.

- Number columns: fill missing with the median, then scale (`StandardScaler`).
- Text columns (`Location`, wind directions, `RainToday`): fill missing with the most common value, then one-hot encode.

Rain days are rare, so training sets `scale_pos_weight` = dry days / rain days. This helps recall reach the 0.75 guardrail.

6.1 Hyperparameters

Defaults for `POST /train`, from `params.yaml` (`model:`). The champion may use other values: `make baseline` picks them by random search. The champion's actual parameters are recorded with its MLflow run.

Table 6.1 Hyperparameters

Parameter	Value
`n_estimators`	250
`max_depth`	4
`learning_rate`	0.05
`subsample`	0.9
`colsample_bytree`	0.9
`random_state`	42 (`config/settings.py`)

6.2 Split

Chronological by whole dates, rolling windows from `params.yaml`. Test is the most recent period.

Table 6.2 Split

Split	Window
Train	up to 1825 days before validation
Validation	90 days
Test	newest 90 days

6.3 Historical metrics (old 70 / 15 / 15 split)

These figures come from the earlier fractional split and do not describe the current champion; the champion's own metrics are stored with its MLflow run (see Chapter 11 for the current results).

Table 6.3 Historical metrics (old 70 / 15 / 15 split)

Metric	Train	Validation	Test
Rows	99,569	21,336	21,337
Accuracy	0.8100	0.8201	0.7896
Precision	0.5549	0.5433	0.5405
Recall	0.7990	0.7336	0.7706
F1	0.6549	0.6243	0.6354
ROC-AUC	0.8922	0.8756	0.8660

How a model gets promoted: Chapter 7 (Model registry and promotion).

6.4 Retrain

```
make train
make train TRAIN_PARAMS='{"n_estimators": 400, "max_depth": 6}'
```

`make train` starts the API if needed, then calls `POST /train` through Nginx.

CHAPTER 7

Model registry and promotion

A new model must not reach users unless it is demonstrably better. This chapter describes how MLflow records models and how the promotion gate decides which version holds the @champion alias.

Models are versioned in **MLflow**, not in Postgres. The serving name is `weather-rainfall-classifier`. The production pointer is the alias `champion`. Promotion happens with `make compare`, or automatically in the Chapter 8 (Orchestration with Airflow) `weather_retrain` DAG (started by the Grafana alert).

The registry is per machine. MLflow keeps it in SQLite under `./mlflow-data/` (git-ignored, not in DVC). A fresh clone has no `@champion`, so the first `make compare` / `make pipeline` promotes the local candidate **and** writes `stage=champion` to the shared Postgres `model_versions`.

Airflow promotions do not update Postgres. The DAG's `compare` task gets no `SUPABASE_*` env, so `@champion` moves in MLflow but `model_versions` keeps the old champion. The mirror is brought back in line with `make reconcile-model-catalog`. Promotion is reserved for the release owner; other team members only serve and query models (`make api`, `make predict`).

`public.model_versions` mirrors releases in Postgres (catalog: training snapshot, stage `candidate` / `champion` / `previous`); MLflow stays the source of truth for the alias.

7.1 What a training run records

Experiment `weather-rainfall-classifier`, one run per training.

Table 7.1 What a training run records

Logged	Content
Params	`n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `random_state`, `train_rows`, `scale_pos_weight`, `negative_count`, `positive_count`
Metrics at training	`train_accuracy`, `train_precision`, `train_recall`, `train_f1`, `train_roc_auc`

Metrics added by `make compare`	`validation_*`, `candidate_recall`, `candidate_roc_auc`
Tags	`stage`, `dataset_sha256`, `parent_sha256`, `git_commit`, `preprocessing_version`
Artifacts	`model/`, `joblib/rain_classifier.joblib`, `dataset/manifest.json`, `lineage/release.json`, `metrics/train.json`
Added by `make validate` / `make evaluate`	`metrics/validation.json`, `metrics/evaluation.json`. Test metrics stay a file, never a run metric

The run registers a new version of the model, tags it with dataset_sha256 and git_commit, and moves @candidate to it. lineage/release.json connects run, version, model hash, dataset hash and commit together.

7.2 First champion

`make baseline` bootstraps the empty alias once: random search (12 draws, `params.yaml` `baseline.search`) on a frozen Kaggle validation window, refit on all Kaggle rows, evaluate on the untouched test window, then `promote_baseline_champion` sets `@champion`. Later candidates go through `make compare`.

To start over before a new baseline, `scripts/reset_prebaseline.py` archives and clears model lineage: MLflow runs, catalog tables (`predictions`, `drift_reports`, `model_versions`, `dataset_versions`, `ingestion_batches`) and local model, metric and derived data files. Kaggle data and `weather_observations` stay. The reset is meant to be run with `--backup-dir`, and first with `--dry-run`; the real run also needs `--confirm RESET_PREBASELINE_EXPERIMENTS`.

7.3 Lifecycle

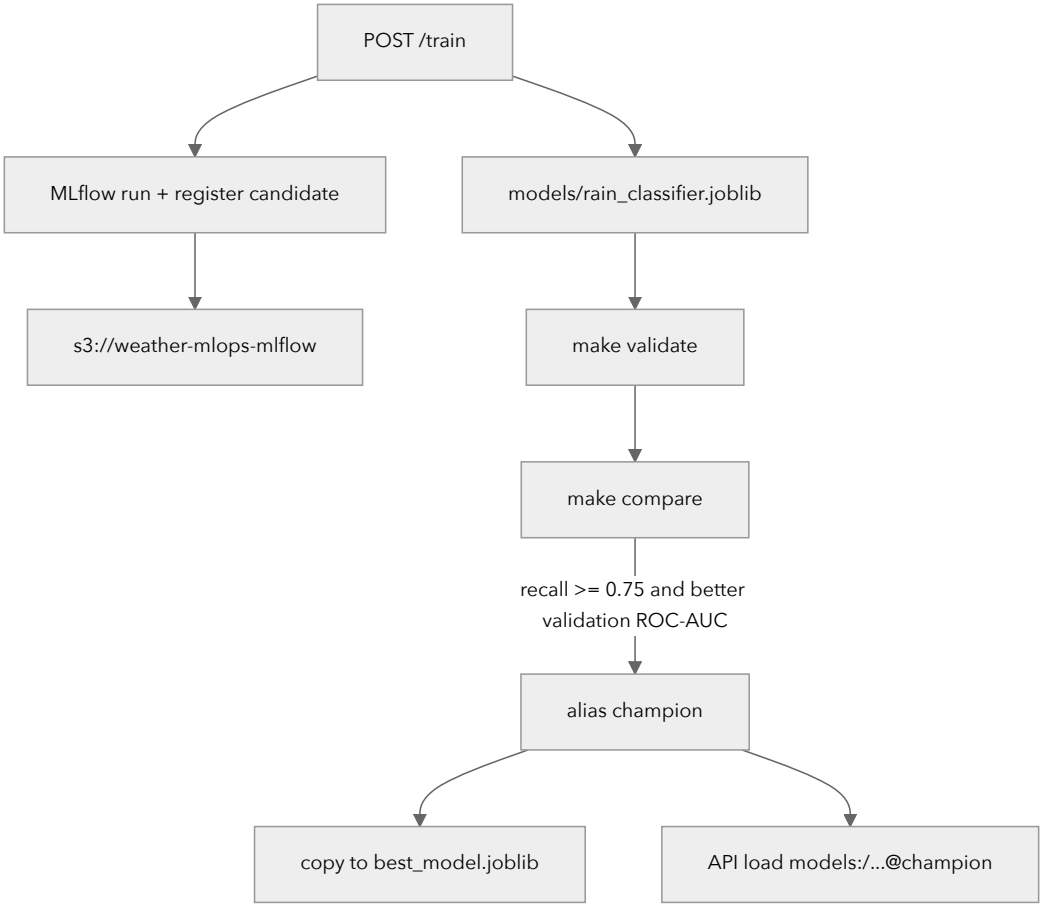


Figure 7.1 Lifecycle

Table 7.2 Lifecycle of a model version: steps and outputs

Step	Command / call	What it writes
Train	`make train` → `POST /train`	local joblib, `reports/metrics/train.json`, MLflow run
Register	`src/weather_mlops/models/tracking.py`	sklearn flavor + joblib + `lineage/release.json` + `dataset/manifest.json` (sha256, git commit)
Validate	`make validate`	`reports/metrics/validation.json` bound to model/dataset/run hashes
Promote	`make compare`	moves alias `champion`; copies `best_model.joblib`
Evaluate	`make evaluate`	test metrics of the **latest candidate** (`rain_classifier.joblib`), even if compare rejected it
Serve	`predict.py` `load_model()`	registry first, then `best_model.joblib`, then `rain_classifier.joblib`

7.4 Promotion rule

`make compare` uses **validation** metrics:

1. `validation_recall` ≥ 0.75, always.
2. First candidate with that recall becomes `champion`.
3. Later candidates replace `champion` only if they beat the current `validation_roc_auc` **and** keep recall ≥ 0.75.
4. If the champion run has no `validation_roc_auc`, compare stops with an error. It never treats a missing score as 0.

Test metrics from `make evaluate` are a report, not the promotion gate.

7.5 What the API returns

`GET /health` and `POST /predict` include a `model` object: alias + version. `/health` answers without asking MLflow, so the Docker healthcheck stays fast. The API caches the model by champion version: after a promotion, the next request loads the new model.

7.6 DVC copy

DVC tracks only the served model, `models/best_model.joblib.dvc`. Candidates (`rain_classifier.joblib`) stay local and in MLflow.

7.7 Source files

Table 7.3 Source files (Model registry and promotion)

Concern	File
Log + register	`src/weather_mlops/models/tracking.py`
Release check	`src/weather_mlops/models/release.py`
Alias move	`src/weather_mlops/models/comparison.py`
Validation / test reports	`src/weather_mlops/models/evaluation.py`
Baseline runs	`src/weather_mlops/models/baseline.py`
Load champion	`src/weather_mlops/models/predict.py`
Names / thresholds	`src/weather_mlops/config/settings.py` (`mlflow_*`)

CHAPTER 8

Orchestration with Airflow

Airflow turns the pipeline into a daily, unattended process and runs retraining on demand. This chapter describes the two DAGs, their schedule and catch-up behaviour, and the security boundary around the Docker socket.

Airflow is a separate Compose stack under `airflow/`. It holds two DAGs: `weather_pipeline` (daily, 18:00 Europe/Berlin) watches the data and the model; `weather_retrain` (no schedule) retrains, and is started by the Grafana retraining alert or by hand. Not part of `make up` — bring it up with `make airflow-up`.

8.1 Components

Table 8.1 Files of the Airflow stack

Piece	Path
Compose	`airflow/docker-compose.yml`
DAGs	`airflow/dags/weather_dag.py`

Table 8.2 Schedule, catch-up and retries of the two DAGs

DAG	Schedule	Catch-up	Retries
`weather_pipeline`	`0 18 * * *` Europe/Berlin, from 2026-09-24	on, one run at a time	3, 1 min apart
`weather_retrain`	none (alert or by hand)	off	3, 1 min apart

8.2 Catch-up

If the laptop was off, `weather_pipeline` replays every missed day, oldest first (`catchup=True`, `max_active_runs=1`). Each run fetches its own date: ingestion gets `OPEN_METEO_DATE` = the day before the run's logical date. In clock time, the run firing at 18:00 on day D fetches day D-2 (so `RainTomorrow` for that day is already known). A missed week fills in without gaps.

8.3 DAG stages

```
weather_pipeline:  ingestion → preprocess → register_dataset → reconcile_outcomes
→ evidently_monitor
weather_retrain:   train → evaluation → compare
```

Table 8.3 Tasks of the daily weather_pipeline DAG

DAG	Task	Image	Command
`weather_pipeline`	ingestion	`weather-ingestion:latest`	entrypoint
`weather_pipeline`	preprocess	`weather-preprocess:latest`	entrypoint
`weather_pipeline`	register_dataset	`weather-api:latest`	`python3 scripts/register_dataset_version.py --kind all --created-by airflow --source open-meteo`
`weather_pipeline`	reconcile_outcomes	`weather-api:latest`	`python3 -m weather_mlops.data.outcomes`
`weather_pipeline`	evidently_monitor	`weather-api:latest`	`python3 -m weather_mlops.monitoring.drift` (Pushgateway + MLflow + Postgres)

Table 8.4 Tasks of the on-demand weather_retrain DAG

DAG	Task	Image	Command
`weather_retrain`	train	`weather-api:latest`	`python3 scripts/train_via_api.py`
`weather_retrain`	evaluation	`weather-api:latest`	`python3 -m weather_mlops.models.evaluation --x-data data/processed/X_validation.csv --y-data data/processed/y_validation.csv --metrics-output reports/metrics/validation.json --split-name validation`
`weather_retrain`	compare	`weather-api:latest`	`python3 -m weather_mlops.models.comparison`

`weather_retrain` is triggered by the Grafana contact point `airflow-retrain` (`POST /api/v1/dags/weather_retrain/dagRuns`, conf `{"triggered_by": "grafana"}`), at most once per 24h. See Chapter 9 (Monitoring).

Training registers a new version under `@candidate`; comparison reads `models:/weather-rainfall-classifier@champion` in MLflow. See Chapter 7 (Model registry and promotion).

8.4 Security boundary

The DAG runs against a docker-socket-proxy, not the host Docker socket. The proxy only allows: starting/stopping containers (`CONTAINERS=1`, `POST=1`) and reading local images (`IMAGES=1`). `EXEC`, `VOLUMES`, and `NETWORKS` are off. It narrows the API, but a task can still bind-mount host folders, so treat Airflow access like host access. Allowlist is in `airflow/docker-compose.yml` (`docker-socket-proxy` service).

8.5 Networks

The Airflow stack runs on `airflow_network`. The app stack runs on `jul26_bmlops_int_weather_weather_network`. `airflow-scheduler` sits on **both** so DAG containers reach the API; `airflow-webserver` is on both too, so Grafana can call the Airflow REST API. Each task container joins `weather_network` through its `network_mode` setting in `weather_dag.py`.

8.6 Running the stack

The stack has the following prerequisites:

- Clone folder is named `jul26_bmlops_int_weather`. The DAG hardcodes the network `jul26_bmlops_int_weather_weather_network`, which Compose names after the folder.
- `make up` has run once. It creates that network and builds the `weather-ingestion` / `weather-preprocess` images the DAG uses (`make api` alone does not).
- `make monitoring` is running. `evidently_monitor` pushes to Pushgateway and fails without it. `weather_retrain` also needs Nginx and MLflow.
- Vault (or the root `.env`) holds `AIRFLOW_API_USER`, `AIRFLOW_API_PASSWORD`, `GF_SECURITY_ADMIN_USER`, `GF_SECURITY_ADMIN_PASSWORD`. `make airflow-up` stops if one is missing. Add one with `make vault-put NAME=... VALUE=...`.

```
cp airflow/.env.example airflow/.env
# fill all four: POSTGRES_ADMIN, POSTGRES_PW, POSTGRES_DB, FERNET_KEY
uv run python -c "from cryptography.fernet import Fernet;
print(Fernet.generate_key().decode())"
make up && make monitoring
make airflow-up               # stop: make airflow-down
```

UI: `http://127.0.0.1:8081`. Log in with `AIRFLOW_API_USER` / `AIRFLOW_API_PASSWORD` (from Vault). Both DAGs start paused; they are unpaused in the UI or from the command line:

```
cd airflow
docker compose --env-file .env --env-file ../.env exec airflow-scheduler airflow
dags unpause weather_pipeline
docker compose --env-file .env --env-file ../.env exec airflow-scheduler airflow
dags unpause weather_retrain
```

The first unpause on a fresh Airflow DB replays every day since 2026-09-24 (Section 8.2). `make airflow-reset` wipes the Airflow DB (`down -v`), so the next unpause replays all those days again.

When a run fails, the task log in the UI gives the cause; the most common one is the Open-Meteo timeout — see Chapter 4 (Training and serving pipeline).

8.7 Local-only secrets

`airflow/.env` is personal: Airflow's own Postgres login and `FERNET_KEY` (the key Airflow uses to encrypt stored passwords). Not in Vault. The Airflow admin login is in Vault.

8.8 What Airflow does not do yet

- Public HTTPS exposure. Airflow runs on `127.0.0.1:8081` only.

8.9 Source files

Table 8.5 Source files (Orchestration with Airflow)

Concern	File
DAG definition	`airflow/dags/weather_dag.py`
Compose stack	`airflow/docker-compose.yml`
Makefile targets	`airflow-up`, `airflow-down`, `airflow-reset` in root `Makefile`
Env template	`airflow/.env.example`

CHAPTER 9

Monitoring

Monitoring closes the loop: it measures the service, the data and the model's real performance, and it is the only component that can start retraining. This chapter describes the metrics, the Evidently job, the alert rules and the dashboards.

Prometheus collects, Grafana shows and alerts, Evidently judges data and delayed performance. Only one alert retrains: delayed performance on real outcomes. All other alerts are shown in Grafana for an operator to act on.

```
make monitoring          # MLflow + API + Nginx + Prometheus + Pushgateway +
Grafana
make evidently           # host run, writes reports/evidently/feature_drift.html
make evidently ARGS='--simulate'   # inject loud synthetic drift
make evidently ARGS='--current-path data/raw/weatherSingapore_current.csv'
```

Prometheus `127.0.0.1:9090`, Pushgateway `127.0.0.1:9091`, Grafana `127.0.0.1:3000`. Grafana credentials and the Airflow API user come from Vault (`with_vault_env.py --monitoring`).

`make evidently` needs logged predictions (run `make predict` first) and `data/processed` (`make dvc-pull`). Every run, even `--simulate`, writes a `drift_reports` row and overwrites the shared `evidently/latest/` report.

9.1 Flow

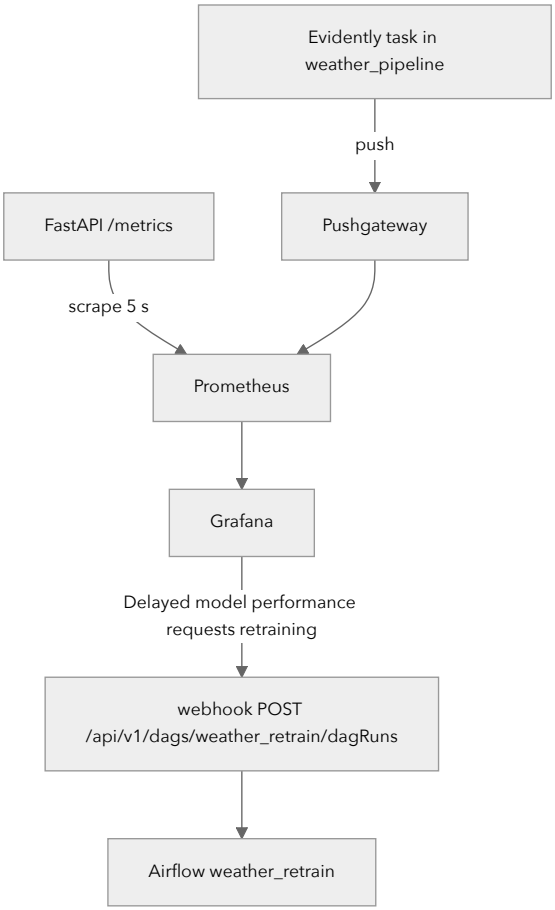


Figure 9.1 Flow

9.2 Metrics

Table 9.1 Metrics

Source	Metrics
API (`api/metrics.py` + instrumentator)	`http_requests_total` , `http_request_duration_seconds` , `model_predictions_total{location,rain_tomorrow}` , `model_probability_distribution` , `model_training_total` , `model_training_duration_seconds` , `model_info` , `model_performance{metric}`
Evidently (`monitoring/metrics.py` → Pushgateway)	`weather_evidently_feature_drift_detected` , `weather_evidently_drifted_column_share` , `weather_model_labeled_predictions` , `weather_model_delayed_recall` , `weather_model_delayed_roc_auc` , `weather_model_retrain_requested` , `weather_evidently_report_timestamp_seconds`

Nginx answers `/metrics` with 404 from outside. Prometheus reads it inside `weather_network` .

9.3 Evidently job

`python -m weather_mlops.monitoring.drift` runs as the last task of the daily `weather_pipeline` DAG, after `reconcile_outcomes` has filled `observed_label` in `predictions` with the real next-day weather. Reconcile only looks at predictions that still have no label. The job fetches up to 5000 logged prediction rows (all time, no date filter). Drift is computed on a sample of up to 4000 rows from each side (served features and training split); delayed performance uses all labelled rows.

Table 9.2 Evidently checks: drift warning and retraining rule

Check	Rule	Effect
Feature drift	5 columns (`Location` , `MinTemp` , `Humidity3pm` , `Pressure3pm` , `RainToday`), PSI threshold 0.1, drift if ≥ 30% of columns drift	Warning only. Never retrains on its own
Delayed performance	needs ≥ 30 labelled predictions; retrain if recall < 0.75, or if delayed ROC-AUC is more than 0.02 below the champion's validation ROC-AUC (recall only when the champion score cannot be read)	Sets `weather_model_retrain_requested = 1`

Each run writes to three places:

Table 9.3 Where each Evidently run writes its results

Where	What
Postgres `drift_reports`	one `feature_drift` row, plus a `performance` row when there are enough labels
MLflow experiment `weather-monitoring`	one run: metrics, HTML report, `summary.json` , tags (`dataset_sha256` , serving model version)
`s3://weather-mlops-mlflow/evidently/latest/`	copy of the newest HTML report, overwritten each run

The monitoring experiment is separate, so Evidently runs never look like training runs. If MLflow or Storage is down, the report still finishes.

9.4 Alerts

Provisioned in `docker/grafana/provisioning/alerting/alerts.yml` .

Seven alerts. Only **Delayed model performance requests retraining** acts: it calls the `airflow-retrain` webhook, at most once per 24h. The other six show in the Grafana UI only: API down, API 5xx rate, API latency, model ROC-AUC below baseline, model recall below guardrail, and rain prediction rate unusual (< 5% or > 60% of the last 24h, ≥ 20 predictions).

ROC-AUC / recall alerts read `model_performance`, the validation metrics set by the last `/train`. It lives in API memory, so it is empty after an API restart. They go to a person: retraining on the same data would give the same model.

`model_predictions_total` and `model_info` (rain-rate alert, active-model panel) only count `/predict/live_data` (Streamlit), not `/predict`. No chat notification channel is configured yet, to avoid noise.

9.5 Dashboards

`docker/grafana/dashboards/grafana_api.json` (requests, errors, latency) and `grafana_model.json` (rain rate, probabilities, active model, performance, Evidently). Data source, dashboards and alerts are provisioned as code.

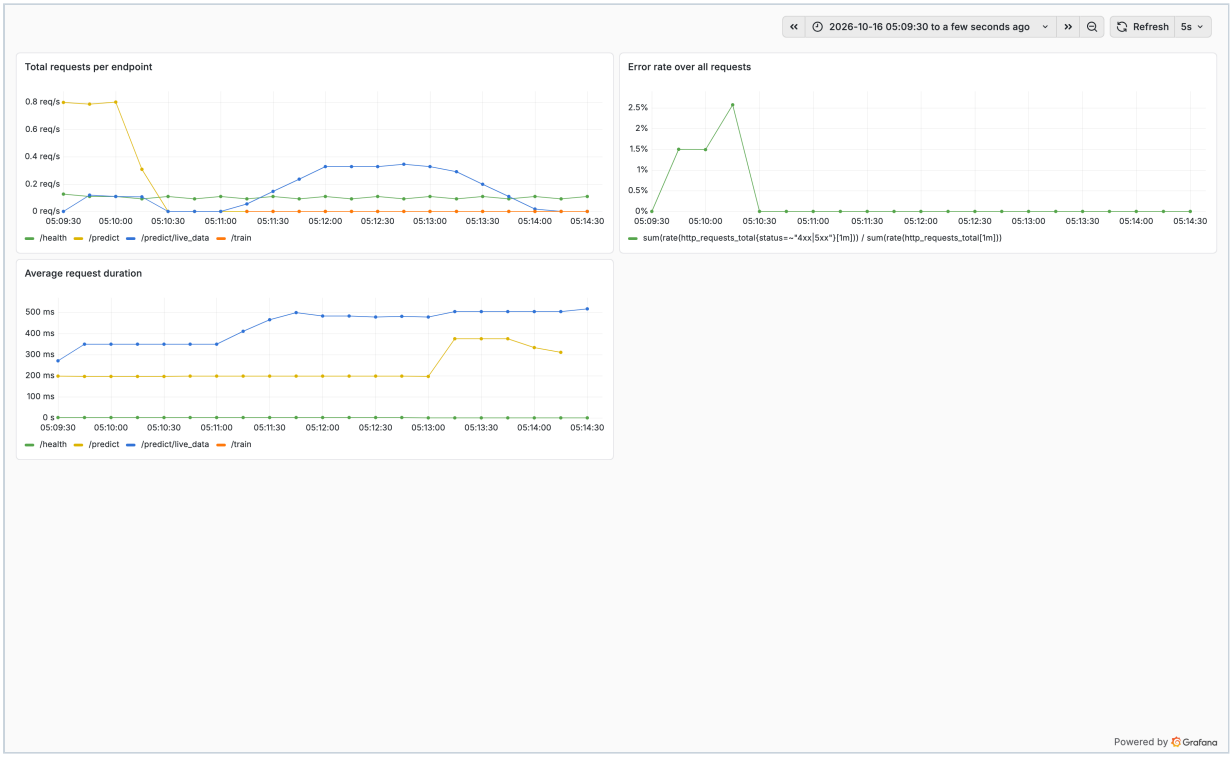


Figure 9.2 Grafana dashboard Weather API, captured on 16 October 2026 from the running stack: requests per endpoint, error rate and average latency. The small error bump is one deliberate request without credentials (HTTP 401).

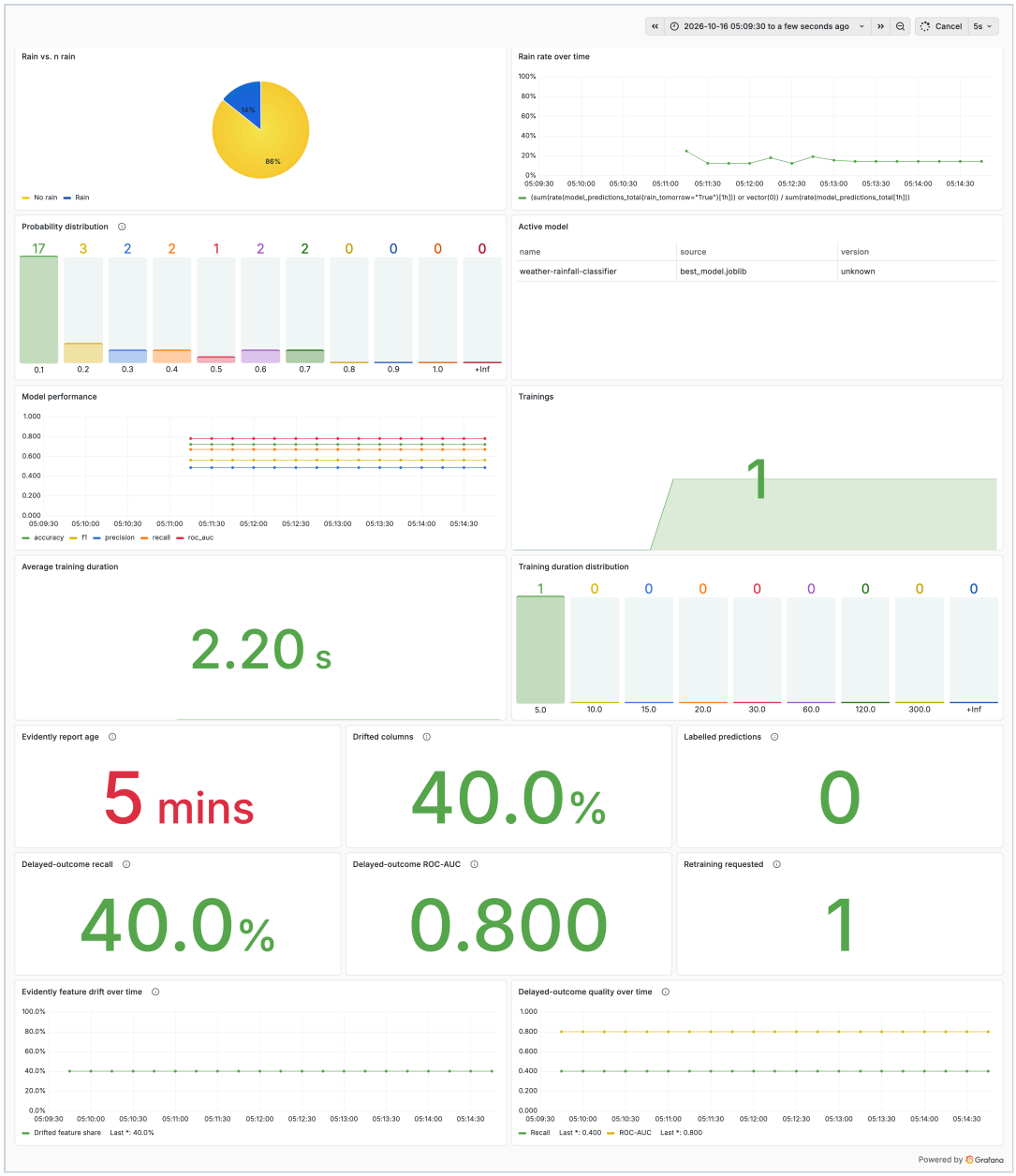


Figure 9.3 Grafana dashboard Weather Model: prediction mix, served model, last training, drift and delayed-outcome quality. The served model is the restored local copy best_model.joblib (this machine's MLflow held no @champion), and the training shown was run without MLflow tracking. Drift (40 %), delayed recall (40 %) and ROC-AUC (0.800) come from the defense simulation POST /monitor/simulate, which injects a feature shift and a preset recall instead of real outcomes; no prediction has a known outcome yet, hence Labelled predictions = 0.

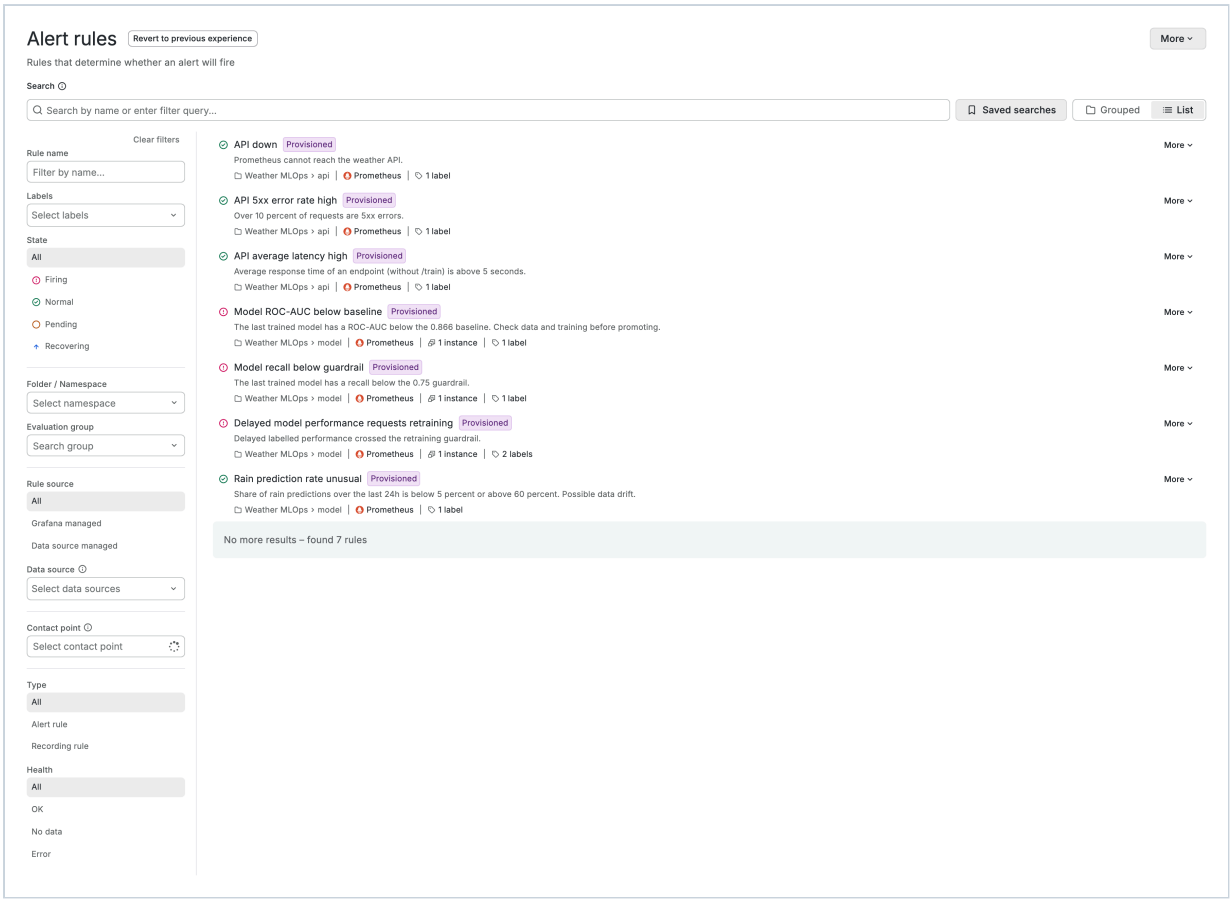


Figure 9.4 Grafana alert rules. Firing: the last training is below the 0.866 ROC-AUC baseline and the 0.75 recall guardrail, and delayed performance requests retraining, which calls the Airflow REST API. The three API rules and the rain-rate rule are normal.

9.6 Source files

Table 9.4 Source files (Monitoring)

Concern	File
API metrics	`src/weather_mlops/api/metrics.py`
Evidently + decision	`src/weather_mlops/monitoring/drift.py`
Pushgateway export	`src/weather_mlops/monitoring/metrics.py`
Singapore out-of-distribution check	`scripts/score_drift.py`
Outcome reconciliation	`src/weather_mlops/data/outcomes.py`
Scrape config	`docker/prometheus/prometheus.yml`
Grafana	`docker/grafana/`

CHAPTER 10

Database schema

Supabase Postgres stores the catalog of datasets and models, every prediction and every monitoring report. This chapter lists the tables and the code that writes each of them.

Canonical SQL: `supabase/schema.sql`. Migrations sit next to it. This chapter maps each table to the code that writes it, rather than reproducing the SQL.

10.1 Tables, buckets and functions

Table 10.1 Tables, buckets and functions

Object	Identity	Written by
`weather_observations`	`(date, location)`	`make load-db` only, by hand (the DAG does not refresh it)
`dataset_versions`	`(dataset_name, sha256)`	`make register-dataset`
`ingestion_batches`	`(batch_date, sha256)`	not yet written: `store_ingestion_batch` exists but is not called
bucket `weather-mlops-dvc`	object key `raw|processed/<sha>/...`	DVC + register-dataset
bucket `weather-mlops-mlflow`	MLflow artifact paths	MLflow `--serve-artifacts`
`public.get_app_secret(name)`	Vault row name	API / scripts at startup
`model_versions`	release per training snapshot	`data/database.py` catalog upsert; stage `champion` / `previous`
`predictions`	one row per served request	API `log_serving_prediction`; `observed_label` filled later by `data/outcomes.py`
`drift_reports`	one row per Evidently run	`monitoring/drift.py` `store_drift_report`

The serving model still comes from the MLflow `@champion` alias, not from `model_versions`. Logging a prediction never fails the request.

10.2 Secrets

Local `.env` needs only `SUPABASE_URL` + `SUPABASE_KEY` (see Appendix A (Setup)). Vault holds the rest: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `API_AUTH_USER`, `API_AUTH_PASSWORD`, `SUPABASE_DB_URL`, `AIRFLOW_API_USER`, `AIRFLOW_API_PASSWORD`, `GF_SECURITY_ADMIN_USER`, `GF_SECURITY_ADMIN_PASSWORD`. Add one with `make vault-put NAME=... VALUE=...`. Row-level security (RLS) is on; the app's service-role key bypasses it.

10.3 Changing the schema

Add a file to `supabase/migrations/`, then `make supabase-migrate` (needs `SUPABASE_DB_URL` in Vault). `make supabase-verify` only checks. Applied files are listed in the `schema_migrations` table.

CHAPTER 11

Results and discussion

This chapter evaluates the system against the objectives of Section 1.2: how good the model is, whether the promotion gate behaves as intended, how the model reacts to a real distribution shift, and what the engineering work delivers.

11.1 Evaluation protocol

All splits are chronological and contain whole dates, so the same day's weather at nearby stations can never appear in two splits. With the current data, the model is trained on five years (March 2014 to March 2019), validated on the next 90 days (March to June 2019) and tested on the following 90 days (June to September 2019). The validation window is used for model selection and for the promotion gate; the test window is used only for the final report and never to choose a model.

Note that validation and test are 100 % Open-Meteo data, while training is 35 % Open-Meteo and 65 % station data. Open-Meteo is a weather model's estimate at a coordinate, not the station's own instruments, so this evaluation also measures a change of data source.

11.2 Model performance

Table 11.1 combines two sources. The champion's figures come from the baseline hyperparameter search, as reported at the defense; the candidate's figures come from the metric files tracked by DVC for the current data snapshot (run `aece6759`), and we reproduced them by retraining with the same seed.

Table 11.1 Model performance

Model	Window	ROC-AUC	Recall	Precision
Majority class ("always no")	test, 90 days (4,410 rows)	0.500	0.000	–
Champion, baseline settings (search run, before refit)	validation, 90 days (earlier window)	0.866	0.776	0.512
Latest candidate	validation, 90 days (4,410 rows)	0.780	0.668	0.485

Latest candidate	test, 90 days (4,410 rows)	0.815	0.703	0.502

The majority-class baseline is 74.7 % accurate on the test window with a recall of zero. With its baseline settings, the champion reached a ROC-AUC of 0.866 on validation days that the search had not trained on, with a recall above the 0.75 guardrail. Its precision of about 0.51 is the price we accept: a missed rainy day costs more than a false alarm, so the gate protects recall and lets precision follow.

Two champion scores appear in this report and they measure different things. The 0.866 above is the baseline settings scored on 90 unseen validation days during the search; the champion that was then refit on all Kaggle rows carries a stored gate score of about 0.89, measured partly on days it had seen (Section 12.1). The gate compares candidates against that stored score.

The promotion gate works in practice. The latest candidate reaches a validation recall of only 0.668, below the 0.75 guardrail, so the gate rejects it and the champion keeps serving without any manual step (an earlier candidate shown at the defense was rejected at 0.673 for the same reason). Its test ROC-AUC of 0.815 also shows that the newer, Open-Meteo-based windows are harder than the Kaggle-era data the champion was tuned on.

11.3 Distribution shift: Australia to Singapore

To test the system against a real distribution shift, we applied the Australian model to Singapore, which uses the same data format but where 82.6 % of days are followed by rain, compared with 21.9 % of all rows in the Kaggle data (22.4 % of the rows that carry a label). Evidently's PSI test flagged all five monitored features.

Table 11.2 Distribution shift: Australia to Singapore

Experiment	ROC-AUC
Australian model, all Singapore days (2014–2024)	0.696
Australian model, Singapore 2022–2024	0.641
Retrained on Singapore 2014–2021	0.703
Retrained on Australia + Singapore, scored on Singapore	0.708
Retrained on Australia + Singapore, scored on the Australian test set	0.867

Retraining recovers only part of the loss, and the combined model keeps its Australian quality. This is the main argument for the monitoring loop: a model that was good at deployment does not stay good when the weather it sees changes.

11.4 Engineering results

- **Tests:** 288 pytest tests, of which 282 unit and configuration tests run on every push and 6 live tests run against the real Nginx container (redirect, TLS, HSTS, health, 401 and 429).
- **Continuous integration:** two jobs on every push and pull request to `master` or `main` (linting, formatting and tests; image builds plus the full suite in the test container). On `master`, a release workflow pushes the `ingestion`, `preprocess` and `api` images to the GitHub Container Registry.
- **Monitoring:** two Grafana dashboards and seven alert rules, one of which starts retraining through the Airflow REST API, at most once every 24 hours.

CHAPTER 12

Limitations and future work

No system of this scope is complete. This chapter lists the limitations we know of, each with the fix we would apply, followed by the next steps we would take beyond the bootcamp.

12.1 Known limitations

We list the limitations we know of, each with the fix we would apply.

Table 12.1 Known limitations

Limitation	Fix
Open-Meteo data stop in September 2019; the years after are not backfilled yet	Finish the resumable backfill (`make backfill-open-meteo`)
Open-Meteo is a modelled estimate, not a station gauge, and counts days from midnight instead of 9:00	Monitor the shift with PSI and train on mixed data
Fixed decision threshold of 0.5; precision is about 0.51	Tune the threshold on the validation window
The champion's gate score (about 0.89) was measured on its own, older window, partly on days it had seen in training, so the bar for candidates is too high	Re-score the champion on the candidate's validation window before comparing
New dataset versions are registered automatically, but blobs are pushed to DVC by hand	Add `dvc push` as a task of the daily DAG
The retraining DAG has no test-set report	Add the evaluate step after compare
Delayed performance is computed without a time window	Use a rolling window, for example the last 30 days
Model counters in Prometheus are updated only by `/predict/live_data`	Increment them in `/predict` as well
Single host, SQLite MLflow backend, self-signed certificate, Basic auth, training inside the API process	Kubernetes, a Postgres backend for MLflow, a real certificate, per-client API keys and a separate training worker

12.2 Future work

- **Scale on Kubernetes:** run resilient API replicas and add capacity as live traffic grows.

- **Jev and transformer models:** compare Jev (TypeSafe [10]) and transformer-based forecasters with the current XGBoost baseline, and check whether better forecasts justify the higher inference cost.
- **Canary release:** send 10 % of live traffic to the candidate and promote it only when its real recall holds.
- **Shadow evaluation and rollback:** score candidates beside the champion without returning their predictions to users, monitor real recall and move the champion alias back to the previous version when it drops.

CHAPTER 13

Conclusion

We built a system that predicts next-day rain at 49 Australian stations and keeps that prediction under control after deployment. Every model can be traced to its data through SHA-256 manifests, DVC and MLflow; every promotion goes through an explicit gate; every prediction is served through one authenticated gateway and stored with the model version that produced it; and every evening the system compares its predictions with the real weather.

The results support the design choices. Accuracy would have rewarded a model that never predicts rain, so we promote on ROC-AUC and protect recall. The gate has already rejected a weaker candidate without human intervention. The Singapore experiment shows that a shift in the data can cost a large part of the model's quality, and that retraining alone does not recover it all, which is why monitoring continues after deployment.

The main open points are the missing years of data after 2019 and the move from a single host to a production platform. Both are listed with concrete next steps in Chapter 12.

References

1. J. Young and A. Young. *Rain in Australia* (weatherAUS dataset), Kaggle. https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package. Original observations: Australian Bureau of Meteorology, http://www.bom.gov.au/climate/data.
2. Open-Meteo. *Historical Weather API*. https://open-meteo.com/en/docs/historical-weather-api. Data licensed under CC BY 4.0 (attribution: "Weather data by Open-Meteo.com").
3. T. Chen and C. Guestrin. "XGBoost: A Scalable Tree Boosting System." *Proceedings of KDD*, 2016.
4. F. Pedregosa et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research* 12, 2011.
5. MLflow documentation (tracking, model registry, aliases). https://mlflow.org/docs/latest/
6. DVC documentation (data versioning, remotes). https://dvc.org/doc
7. Apache Airflow documentation. https://airflow.apache.org/docs/
8. Evidently documentation (data drift presets, PSI). https://docs.evidentlyai.com/
9. Prometheus documentation, https://prometheus.io/docs/; Grafana documentation, https://grafana.com/docs/
10. TypeSafe. https://typesafe.ai

APPENDIX A

Setup

A.1 Requirements

- Python 3.13 (`.python-version`)
- `uv`
- Docker / Docker Compose (OrbStack or Docker Desktop)
- DVC on the host (`uv` env is enough)

A.2 Install

```
git clone https://github.com/DataScientest-Studio/jul26_bmlops_int_weather.git
cd jul26_bmlops_int_weather
uv sync
cp .env.example .env
```

Fill `SUPABASE_URL` and `SUPABASE_KEY` (the secret / service-role key — ask the project owner). Do not commit `.env` . Every other secret (S3 keys, API login, Airflow and Grafana admins) comes from Supabase Vault.

Delete or comment out the four `AIRFLOW_API_*` / `GF_SECURITY_ADMIN_*` lines. A value in `.env` wins over Vault, so the `<...>` placeholders would become the real logins.

```
make dvc-config
make dvc-pull
make test
make api
curl -k https://127.0.0.1/health
```

Table A.1 Install

Want	Command
Public HTTPS + rate limits	`make api` / `make up`
MLflow UI	`http://127.0.0.1:8080` after `make mlflow` or `make up`

Full demo stack (API, MLflow, monitoring, Airflow, Streamlit)	`make streamlit` → `http://127.0.0.1:8501`
Scheduled DAG	`make airflow-up` → `http://127.0.0.1:8081` – see Chapter 8 (Orchestration with Airflow)
Stop	`make down`

The API port is not opened on the host; go through Nginx: `https://127.0.0.1/docs`. The certificate is self-signed, so the browser warns and `curl` needs `-k`. The complete command table is in the repository README.

A.3 Airflow

Optional. Needs extra env files and running services — follow Chapter 8 (Orchestration with Airflow).

APPENDIX B

Reproduction

Fresh clone, restore the published baseline, then optionally retrain.

```
git clone https://github.com/DataScientest-Studio/jul26_bmlops_int_weather.git
cd jul26_bmlops_int_weather
uv sync
cp .env.example .env          # fill SUPABASE_*, remove the 4 admin lines (see
Setup)
make dvc-config
make dvc-pull
make test
make api
curl -k https://127.0.0.1/health
```

Retrain through Nginx:

```
make pipeline                 # train → validate → compare → evaluate
                              # fresh clone: promotes to the shared catalog (Ch.
7)
make predict
```

For the daily scheduled version, run `make airflow-up` — see Chapter 8 (Orchestration with Airflow).

Refresh Open-Meteo and rebuild splits:

```
make up                       # ingestion → preprocess → API → Nginx
make register-dataset
```

`make up` fetches the day two days ago by default (`RainTomorrow` needs the next day's rain). For another day: `OPEN_METEO_DATE=YYYY-MM-DD make up`. `make register-dataset` writes to the shared Storage bucket and `dataset_versions`.

Publish a new baseline — when you intend the local artifacts to become the team restore point:

```
make dvc-repro
make pipeline
make register-dataset
uv run dvc add data/raw/incremental models/best_model.joblib
make dvc-commit
make dvc-push
git add dvc.lock data/raw/incremental.dvc models/best_model.joblib.dvc
```

`make dvc-add-model` is not used at this step, because it still adds the old `rain_classifier.joblib` + `train.json` pointers.

Commit the updated DVC pointers with the code. Verify from an empty `data/` and `models/` with `make dvc-pull`.

B.1 CI

On push and PRs to `master` / `main`:

- Host: `uv sync --locked`, `ruff check` + `ruff format --check`, pytest (skips tests marked `gateway`)
- Docker: build images, start API + Nginx, run unit + live HTTPS / rate-limit tests

Release workflow pushes images to `ghcr.io` from `master`.

APPENDIX C

Glossary

Table C.1 Glossary

Word	Meaning
DAG	Airflow's scheduled chain of tasks
champion	MLflow alias (movable name) for the model in production
catalog	Postgres tables that record which data/model versions exist
Vault	Supabase secret store; the app loads secrets from it at start
Pushgateway	where batch jobs push metrics so Prometheus can read them
PSI	Population Stability Index: how much a column's values shifted
profile	Compose services that only start when asked (`make monitoring` , ...)
DVC	Data Version Control: versions large data files outside Git, with small pointer files in Git
HSTS	HTTP Strict Transport Security: tells browsers to use HTTPS only
okta	Unit of cloud cover: eighths of the sky covered
precision	Share of predicted rainy days that were actually rainy
recall	Share of actual rainy days that the model predicted as rainy
RLS	Row-level security: Postgres rules on which rows a role may read or write
ROC-AUC	Area under the ROC curve: how well the model ranks rainy above dry days, across all thresholds (0.5 = random)